

Correction — Exercice N°1

Classification de Flynn & Loi d'Amdahl

1. Classification de Flynn

a) Les quatre catégories

Catégorie	Instructions	Données	Description
SISD	1	1	Modèle séquentiel classique (Von Neumann). Un processeur, un flux.
SIMD	1	N	Une instruction appliquée simultanément à N données. Ex. : GPU, SSE/AVX.
MISD	N	1	N instructions sur la même donnée. Rare (tolérance aux fautes, pipeline strict).
MIMD	N	N	Chaque unité exécute ses propres instructions sur ses propres données. Ex. : multi-cœurs, clusters.

b) Catégories permettant une accélération

SIMD et **MIMD** peuvent améliorer les performances. MISD est théoriquement parallèle mais sans gain généraliste. SISD est strictement séquentiel.

c) Type de parallélisme exploité

- **SISD** : Aucun parallélisme — un seul flux d'exécution.
- **SIMD** : Parallélisme de données — même opération sur un vecteur de valeurs.
- **MISD** : Parallélisme de tâches sur une seule donnée (redondance, pipeline strict).
- **MIMD** : Parallélisme de tâches et de données — threads/processus indépendants.

2. Loi d'Amdahl : dérivation

a) Temps d'exécution sur N unités

Soit T le temps séquentiel de référence, p la fraction parallélisable et (1-p) la fraction séquentielle :

$$T(N) = (1 - p) \cdot T + (p \cdot T) / N$$

b) Accélération S(N)

En divisant T par T(N) :

$$S(N) = T / T(N) = 1 / [(1 - p) + p/N]$$

c) Vérification

- Pour $N = 1$: $S(1) = 1/(1-p+p) = 1$ ✓ (aucune accélération sans parallélisme)
- Pour $N \rightarrow \infty$: $S \rightarrow 1/(1-p)$ ✓ (borne supérieure finie si $p < 1$)

3. Interprétation des cas limites

a) Analyse de $p = 0$, $p = 1$, $N = \infty$

Cas	Résultat	Interprétation
$p = 0$	$S(N) = 1$	Programme entièrement séquentiel : aucun gain quelle que soit N .
$p = 1$	$S(N) = N$	Programme entièrement parallélisable : accélération linéaire (idéale).
$N \rightarrow \infty$	$S \rightarrow 1/(1-p)$	Même avec ∞ cœurs, l'accélération est bornée par la partie séquentielle.

Exemple : pour $p = 0,9 \rightarrow S_{\text{max}} = 1/0,1 = 10\times$, même avec un million de cœurs.

b) Lien avec Flynn

- **SISD** $\leftrightarrow p = 0$: processeur séquentiel, $S(N) = 1$ en permanence.
- **SIMD** $\leftrightarrow p$ élevé sur la partie vectorisable, plafonné par les sections scalaires.
- **MIMD** $\leftrightarrow$ modèle général, p dépend de la fraction exécutable sur plusieurs cœurs indépendants.

4. Lien entre Flynn et Amdahl

a) Évaluation des gains par architecture

Architecture SIMD

N = largeur du registre vectoriel. Le gain est borné par $1/(1-p)$ où p est la fraction vectorisable. Si 20 % du code est scalaire : $S_{\text{max}} = 1/(1-0,8) = 1/0,2 = 5\times$, même avec des registres 512 bits.

Architecture MIMD

N = nombre de cœurs/nœuds. Les sections critiques, barrières de synchronisation et communications constituent la fraction $(1-p)$ qui plafonne l'accélération.

b) Pourquoi SISD ne bénéficie d'aucune accélération

En SISD, $N = 1$ par définition architecturale. Substituer $N = 1$ dans la formule donne **$S(1) = 1$** quel que soit p . Il n'existe aucune unité de calcul supplémentaire à exploiter.

c) Exemples concrets de limitation

SIMD limité — boucle avec branchements

```
for (int i = 0; i < N; i++) {  
    if (a[i] > 0)    // divergence du SIMD pour les valeurs négatives ou nulles  
        b[i] = sqrt(a[i]);  
}
```

Les éléments négatifs provoquent non-régularité due au branchement conditionnel"

MIMD limité — section critique

```
for (int i = 0; i < N; i++) {  
    double val = compute(i); // parallèle  
    #pragma omp critical  
    { global_sum += val; } // séquentiel : verrou variable partagée  
}
```

Le verrou sur `global_sum` est la fraction $(1-p)$. Avec 16 cœurs et 5 % de temps en section critique, le gain est plafonné à $1/0,05 = 20\times$ — mais la contention le ramène bien en deçà.

5. Champ d'application de la loi

a) Programmes et matériel concernés

La loi s'applique à tout programme dont une fraction fixe est parallélisable, exécuté sur une machine à mémoire partagée ou distribuée disposant de N unités homogènes (cœurs, SIMD lanes, nœuds).

b) Hypothèses implicites

- La fraction p est constante, indépendante de N et de la taille du problème.
- Les N unités sont identiques et parfaitement disponibles.
- La partie parallèle se divise parfaitement en N sous-tâches sans coût de communication.
- Aucun surcoût de démarrage, de synchronisation ni de communication.
- Mémoire illimitée et d'accès uniforme.

c) Situation où Amdahl surestime l'accélération

Exemple : traitement matriciel sur cluster multi-nœuds avec $p = 0,95 \rightarrow S_{\text{max}} \text{ prédit} = 20\times$.

- **Effets mémoire** : quand N augmente, les données débordent du cache. La bande passante DRAM ne se divise pas par N .
- **Coût des interconnexions** : les échanges MPI entre nœuds (`MPI_Sendrecv`) introduisent une latence croissante avec N .
- **Synchronisation** : `MPI_Barrier` ou `#pragma omp barrier` force tous les threads à attendre le plus lent (load imbalance).
- **Surcoût de démarrage** : pour de petits problèmes, le lancement des threads peut dépasser le gain de parallélisme.

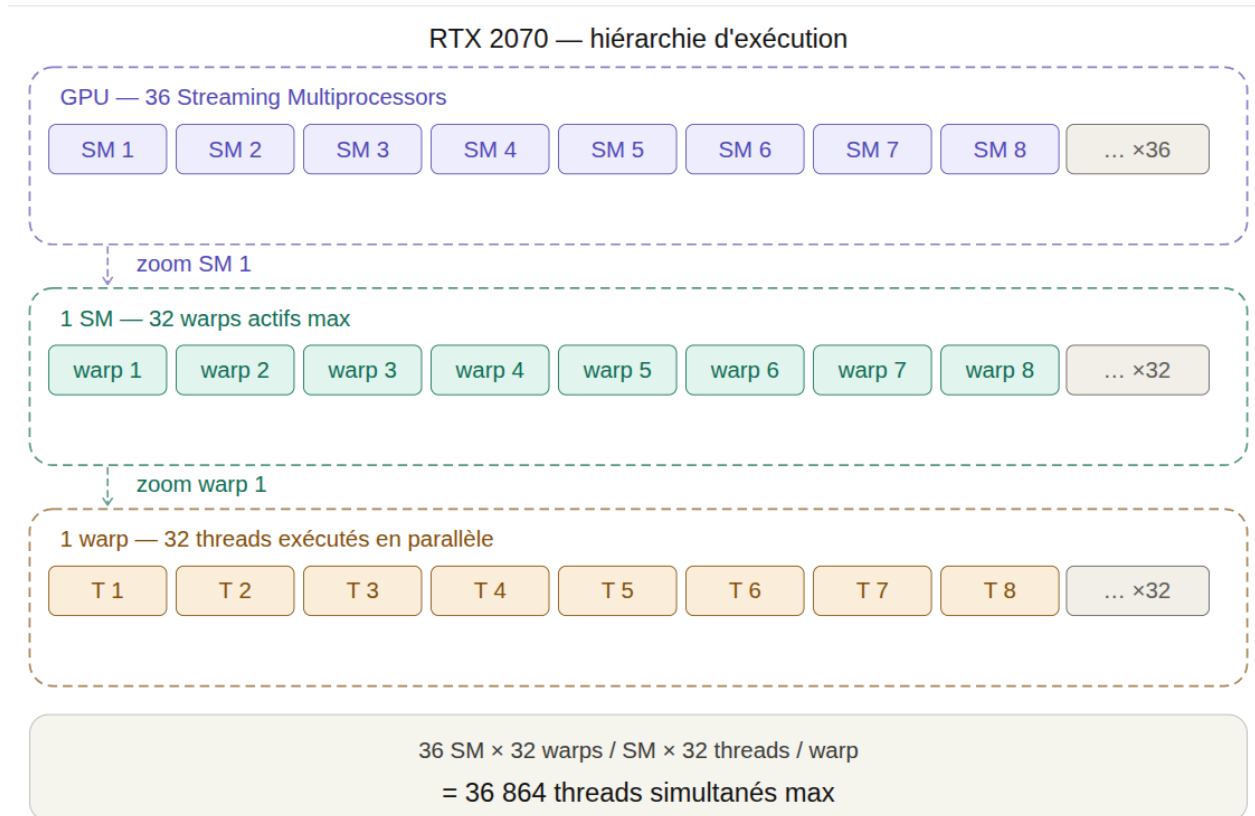
Résultat typique : gain réel de $8\times$ là où Amdahl prédit $20\times$. Pour de très grandes valeurs de N , l'accélération peut même décroître. La loi d'Amdahl est une **borne supérieure optimiste**, non une prédiction réaliste.

La loi d'Amdahl formalise un invariant fondamental : toute portion séquentielle bride irrémédiablement l'accélération, indépendamment de la puissance du matériel parallèle.

Correction exercise N°2

Loi d'Amdahl : Xeon Phi vs RTX 2070

Rappel



Données

	Arch. A — Xeon Phi	Arch. B — RTX 2070
Unités N	61 cœurs	$36 \times 1 \times 32 = 1\,152$ threads
p (parallélisable)	0,98	0,98
(1-p) séquentiel	0,02	0,02

Formule

$$S(N) = 1 / [(1 - p) + p / N]$$
$$S_{\text{max}} = 1 / (1 - p) = 1 / 0,02 = 50\times \text{ avec } p=0,02$$

a. Accélération théoriques

Architecture A — N = 61 :

$$S(61) = 1 / [0,02 + 0,98/61] = 1 / [0,02 + 0,01607] = 1 / 0,03607$$

$$S_A \approx 27,72 \times$$

Architecture B — N_GPU = 36 × 1 × 32 = 1 152 :

$$S(1152) = 1 / [0,02 + 0,98/1152] = 1 / [0,02 + 0,00085] = 1 / 0,02085$$

$$S_B \approx 47,96 \times$$

b. Comparaison à la limite théorique S_max = 50 ×

Architecture	N	S(N)	S_max	S/S_max
Xeon Phi (Arch. A)	61	27,72 ×	50 ×	55,4 %
RTX 2070 (Arch. B)	1 152	47,96 ×	50 ×	95,9 %

c. Le Xeon Phi exploite-t-il bien le parallélisme ?

Non. Avec N = 61, il n'atteint que 55,4 % de la limite théorique. La fraction séquentielle (2 %) domine encore le dénominateur : $0,02 + 0,98/61 = 0,0361$, soit la partie séquentielle représente encore 55 % du dénominateur.

Pour approcher S_max = 50×, il faudrait un N très grand. Le GPU avec N = 1 152 rend la fraction parallèle quasi-nulle ($0,00085 \ll 0,02$) et atteint 95,9 % de la limite.

Conclusion : même 2 % de code séquentiel plafonne définitivement le gain à 50×. Le Xeon Phi manque de cœurs pour limiter cet effet ; le GPU y parvient grâce à ses 1 152 threads.

Correction — Exercice N°3

Comparaison de deux versions CMOS — Puissance dynamique et fuite

Données du problème

Grandeur	Version 1	Version 2
Capacité C	C_1	$C_2 = 0,75 C_1$
Tension Vdd	3 V	2 V
Courant de fuite	$I_{fuite} = 0,57 \text{ mA}$	identique
Facteur d'activité α	identique	identique

Formule de la puissance dynamique : $P_{dyn} = \alpha \cdot C \cdot V_{dd}^2 \cdot f$

a) Même puissance dynamique ($P_{dyn1} = P_{dyn2}$)

Rapport de fréquences f_2/f_1

On impose $P_{dyn1} = P_{dyn2}$:

$$\alpha \cdot C_1 \cdot V_{dd1}^2 \cdot f_1 = \alpha \cdot C_2 \cdot V_{dd2}^2 \cdot f_2$$

On simplifie α , puis on substitue $C_2 = 0,75 C_1$:

$$f_2/f_1 = (C_1 \cdot V_{dd1}^2) / (C_2 \cdot V_{dd2}^2)$$

$$f_2/f_1 = (C_1 \times 9) / (0,75 \cdot C_1 \times 4) = 9 / 3 = 3$$

$$f_2/f_1 = 3 \rightarrow \text{la version 2 peut fonctionner 3× plus vite}$$

Gain sur la période d'horloge

La période $T = 1/f$, donc :

$$T_2/T_1 = f_1/f_2 = 1/3 \approx 0,333$$

$$\text{Gain : } T_2 = T_1/3 \rightarrow \text{période réduite de 66,7 \%}$$

b) Même fréquence $f_1 = f_2$

Rapport des puissances dynamiques

$$P_{dyn2}/P_{dyn1} = (C_2 \cdot V_{dd2}^2) / (C_1 \cdot V_{dd1}^2) = (0,75 \times 4) / (1 \times 9) = 3/9 = 1/3$$

$$P_{dyn2}/P_{dyn1} = 1/3 \rightarrow \text{la puissance dynamique est divisée par 3}$$

Puissance statique

On utilise Vdd (à température constante, $P_{fuite} \propto I_{fuite} \cdot V_{dd}$) :

$$P_{fuite,1} = I_{fuite} \times V_{dd1} = 0,57 \times 3 = 1,71 \text{ mW}$$

$$P_{fuite,2} = I_{fuite} \times V_{dd2} = 0,57 \times 2 = 1,14 \text{ mW}$$

Puissance totale

Avec $P_{\text{dyn},1} = 30 \text{ mW}$:

$$P_{\text{dyn},2} = P_{\text{dyn},1} / 3 = 30/3 = 10 \text{ mW}$$

$$P_{\text{tot},1} = P_{\text{dyn},1} + P_{\text{fuite},1} = 30 + 1,71 = 31,71 \text{ mW}$$

$$P_{\text{tot},2} = P_{\text{dyn},2} + P_{\text{fuite},2} = 10 + 1,14 = 11,14 \text{ mW}$$

	Version 1	Version 2
P_{dyn}	30 mW	10 mW
P_{fuite}	1,71 mW	1,14 mW
P_{tot}	31,71 mW	11,14 mW

Gain global en puissance

$$\text{Gain} = (P_{\text{tot},1} - P_{\text{tot},2}) / P_{\text{tot},1} = (31,71 - 11,14) / 31,71$$

$$\text{Gain} \approx 64,9 \% \rightarrow \text{la version 2 consomme } \sim 65 \% \text{ de moins}$$

c) Compromis fréquence vs puissance

Cas a — même puissance, fréquence triplée : la réduction de V_{dd} et C permet de tripler f sans augmenter la consommation. C'est le levier de performance.

Cas b — même fréquence, puissance divisée par 3 : la version 2 consomme $\sim 65 \%$ de moins à fréquence égale. C'est le levier d'efficacité énergétique (autonomie, dissipation thermique).

En pratique : les concepteurs combinent les deux — on augmente modérément la fréquence *et* on réduit la puissance simultanément, en acceptant un compromis intermédiaire. La tension d'alimentation est le paramètre clé : baisser V_{dd} réduit P_{dyn} en V_{dd}^2 .

Conclusion : la réduction de V_{dd} de 3V à 2V avec $C_2 = 0,75 C_1$ permet soit de tripler la fréquence à iso-puissance, soit de diviser la puissance totale par ~ 3 à iso-fréquence.

Correction — Exercice N°5

Loi de Moore — progression géométrique

1. Nature de la suite (Nn)

La loi de Moore stipule que le nombre de transistors **double tous les deux ans**. Chaque période correspond à 2 ans, donc :

$$N(n+1) = 2 \times N_n$$

Le rapport entre deux termes consécutifs est constant (= 2). La suite est donc une **progression géométrique de raison q = 2**.

2. Expression de Nn

Avec $N_0 = 2300$ transistors (en 1971) :

$$N_n = N_0 \times q^n = 2300 \times 2^n$$

3. Calcul de N₅ (10 ans après 1971, soit 1981)

10 ans = 5 périodes de 2 ans → $n = 5$:

$$N_5 = 2300 \times 2^5 = 2300 \times 32 = 73\,600$$

$$N_5 = 73\,600 \text{ transistors (en 1981)}$$

Ordre de grandeur cohérent : l'Intel 8088 de 1979 comptait ~29 000 transistors.

4. Année de dépassement du million de transistors

On cherche le plus petit n tel que $N_n > 1\,000\,000$:

$$2300 \times 2^n > 1\,000\,000$$

$$2^n > 1\,000\,000 / 2300 \approx 434,8$$

$$n > \log_2(434,8) = \ln(434,8) / \ln(2) \approx 8,76$$

Donc $n = 9$ (première valeur entière supérieure à 8,76).

$$\text{Année} = 1971 + 9 \times 2 = 1971 + 18 = 1989$$

On dépasse 1 million de transistors en 1989

Vérification : $N_9 = 2300 \times 2^9 = 2300 \times 512 = 1\,177\,600 > 10^6$ ✓

n	Année	Nn (approx.)	Processeur réel (référence)
0	1971	2 300	Intel 4004
5	1981	73 600	Intel 8086 (~29 k)
8	1987	~589 000	—
9	1989	~1 178 000	Intel 486 (1989, 1,2 M)
10	1991	~2 355 000	—

5. Représentation graphique sur échelle logarithmique

Une progression géométrique $N_n = N_0 \cdot q^n$ devient linéaire en échelle log :

$$\log(N_n) = \log(N_0) + n \cdot \log(q)$$

Sur une échelle logarithmique (axe y en log), la courbe est une **droite de pente $\log(q) = \log(2) \approx 0,301$** par période.

C'est précisément pourquoi la loi de Moore est représentée sur des graphiques semi-log : une droite sur ce type d'échelle signifie une croissance exponentielle constante.

Correction — Exercice N°6

Architecture RISC — accès mémoire LOAD/STORE

a) Raison du choix LOAD/STORE uniquement

Dans une architecture RISC, l'objectif est d'exécuter **une instruction par cycle d'horloge**. Or un accès mémoire est une opération lente et imprévisible (latence variable, bus partagé).

Si les instructions arithmétiques pouvaient accéder directement à la mémoire (comme en CISC), leur durée d'exécution serait irrégulière et variable, ce qui rendrait le pipeline difficile — voire impossible — à maintenir à un débit d'un cycle par instruction.

En limitant les accès mémoire à deux instructions dédiées (**LOAD** : mémoire → registre, **STORE** : registre → mémoire), toutes les autres instructions (ALU, branchement...) n'opèrent **que sur des registres**, avec une durée d'exécution fixe et prévisible.

b) Conséquence sur l'organisation interne du processeur

Cette règle implique un **grand nombre de registres généraux** (typiquement 32 en RISC, contre 8 en x86 classique).

Puisque toutes les opérations passent par des registres, le processeur doit en disposer suffisamment pour :

- minimiser les allers-retours en mémoire (*register pressure*),
- permettre au compilateur d'optimiser l'allocation des variables dans les registres,
- alimenter un pipeline profond sans créer de dépendances (hazards) entre étages.

Conséquence directe : le pipeline RISC est plus simple, plus régulier et plus efficace. Chaque étage traite une instruction de durée fixe, ce qui permet le découpage en phases uniformes (IF / ID / EX / MEM / WB).

Aspect	CISC	RISC
Accès mémoire	Toute instruction peut accéder à la mémoire	Uniquement LOAD et STORE
Opérandes ALU	Registres ou mémoire	Registres uniquement
Nombre de registres	Peu (ex. 8 en x86)	Nombreux (ex. 32 en MIPS/ARM)
Pipeline	Difficile (durées variables)	Efficace (durées fixes)

Optimalité et efficacité du pipeline : durées fixes

Correction — Exercice N°9

Architecture Von Neumann 16 bits — capacité mémoire, cycles et débit

Partie a — Architecture Von Neumann 16 bits / 24 bits / 2,4 GHz

1. Fonctionnement de l'architecture de Von Neumann

L'architecture de Von Neumann repose sur **quatre composants interconnectés par des bus** :

Composant	Rôle
Processeur (CPU)	Exécute les instructions : unité de contrôle (UC) + unité arithmétique et logique (UAL)
Mémoire centrale	Stocke données et instructions dans le même espace adressable
Bus système	Trois bus : adresses (lecture/écriture), données (transfert), contrôle (signaux R/W)
Entrées/Sorties	Périphériques communiquant via le bus ou contrôleurs dédiés

Principe d'exécution (cycle **Fetch – Decode – Execute**) :

1. **Fetch** : le PC (Program Counter) envoie l'adresse de l'instruction sur le bus d'adresses ; la mémoire renvoie l'instruction via le bus de données.
2. **Decode** : l'UC décode l'opcode pour identifier l'opération et les opérandes.
3. **Execute** : l'UAL effectue le calcul ou l'accès mémoire ; le PC est incrémenté.

Caractéristique clé : instructions et données partagent le même bus et le même espace mémoire (contrairement à Harvard). C'est le *goulot d'étranglement de Von Neumann*.

2. Capacité mémoire adressable

Le bus d'adresses est de 24 bits, donc le nombre d'adresses distinctes est :

$$\text{Nb adresses} = 2^{24} = 16\,777\,216$$

Chaque adresse pointe vers 1 octet (convention standard) :

$$\begin{aligned}\text{Capacité} &= 2^{24} \text{ octets} = 16\,777\,216 \text{ octets} \\ &= 16\,384 \text{ Kio} = 16 \text{ Mio} = 0,015625 \text{ Gio} \approx 16 \text{ Mo}\end{aligned}$$

Unité	Valeur	Calcul
Octets	16 777 216	2^{24}
Kio ($\div 2^{10}$)	16 384 Kio	$2^{24} / 2^{10} = 2^{14}$
Mio ($\div 2^{20}$)	16 Mio	$2^{24} / 2^{20} = 2^4$
Gio ($\div 2^{30}$)	0,015625 Gio	$2^{24} / 2^{30} = 2^{-6}$

3. Temps d'un cycle processeur

La fréquence d'horloge est $f = 2,4 \text{ GHz} = 2,4 \times 10^9 \text{ Hz}$. Le temps de cycle est son inverse :

$$T = 1/f = 1 / (2,4 \times 10^9)$$
$$T \approx 0,4167 \text{ ns} \approx 416,7 \text{ ps}$$

Partie b — Traitement de 12 Mio de données

Rappel des données : 12 Mio = $12 \times 2^{20} = 12\,582\,912$ octets. Blocs de 3 octets. Processeur P1 à 2,4 GHz : 4 cycles/lecture + 9 cycles/traitement.

1. Interprétation d'un bloc de 3 octets

3 octets = 24 bits. Exemples concrets :

- Un **pixel RGB** : 1 octet par canal (Rouge, Vert, Bleu) → format d'image non compressé.
- Un **échantillon audio 24 bits** : résolution professionnelle (studio, 1 sample PCM).
- Trois caractères ASCII consécutifs (ex. : code pays « FRA »).

Le cas le plus courant en traitement de signal/image : pixel RGB 24 bits.

2. Nombre de blocs et temps total — Processeur P1 (2,4 GHz)

Nombre de blocs :

$$N_{\text{blocs}} = 12 \times 2^{20} / 3 = 4 \times 2^{20} = 4\,194\,304 \text{ blocs}$$

Cycles par bloc = 4 (lecture) + 9 (traitement) = 13 cycles/bloc

Total de cycles :

$$\text{Cycles}_{\text{total}} = 4\,194\,304 \times 13 = 54\,525\,952 \text{ cycles}$$

Temps total :

$$t_{P1} = \text{Cycles}_{\text{total}} \times T = 54\,525\,952 \times (1/2,4 \times 10^9)$$
$$t_{P1} \approx 22,72 \text{ ms}$$

3. Processeur P2 (1,3 GHz) — blocs de 4 octets, 2 cycles lecture + 5 cycles traitement

a) Temps de traitement de P2

P2 lit 4 octets par bloc. Nombre de blocs pour 12 Mio :

$$N_{\text{blocs}_{P2}} = 12 \times 2^{20} / 4 = 3 \times 2^{20} = 3\,145\,728 \text{ blocs}$$

Cycles par bloc = 2 + 5 = 7 cycles/bloc

Total de cycles :

$$\text{Cycles}_{P2} = 3\,145\,728 \times 7 = 22\,020\,096 \text{ cycles}$$

Temps de cycle P2 = $1/1,3 \times 10^9 \approx 769,2 \text{ ps}$

$$t_{P2} = 22\,020\,096 / (1,3 \times 10^9)$$
$$t_{P2} \approx 16,94 \text{ ms}$$

b) Comparaison des deux processeurs

On calcule le **débit de traitement** (octets traités par seconde) :

$$\text{Débit P1} = 12 \times 2^{20} / 0,02272 \approx 554 \text{ Mo/s}$$

$$\text{Débit P2} = 12 \times 2^{20} / 0,01694 \approx 743 \text{ Mo/s}$$

Critère	P1 (2,4 GHz)	P2 (1,3 GHz)	Vainqueur
Fréquence	2,4 GHz	1,3 GHz	P1
Cycles/bloc	13	7	P2
Nb blocs	4 194 304	3 145 728	P2
Temps total	≈ 22,72 ms	≈ 16,94 ms	P2
Débit	≈ 554 Mo/s	≈ 743 Mo/s	P2

Conclusion : P2 est plus rapide malgré sa fréquence presque deux fois inférieure. La raison : P2 traite des blocs plus grands (4 octets vs 3) en moins de cycles (7 vs 13), ce qui lui confère un bien meilleur débit effectif. La fréquence seule ne caractérise pas la performance — c'est le débit ($\text{IPC} \times f$) qui compte.